SW Engineering CSC648-848 Fall 2025 Milestone 2 Part I

GatorTutors Team 14

10/23/25

Name	Role	Email
William Wong	Team Lead	wwong23@sfsu.edu
James Barr	Frontend Lead	jbarr@sfsu.edu
Ethan Niu	Frontend Engineer	eniu1@sfsu.edu
Uchenna Duru	Backend Lead	uduru2@sfsu.edu
Randy Chen	Backend Engineer	rchen10@sfsu.edu
Cyrus Molina-Perez	Github Maintainer	cmolinaperez@sfsu.edu

Date Submitted	**11/2/25**
Date Revised	**11/ /25**

Table of Contents

SW Engineering CSC648-848 Fall 2025 Milestone 2 Part I
Table of Contents
1. Executive Summary
2. List of main data items and entities
3. Functional Requirements
4. UI Storyboards for each main use case
5. High level Architecture, Database Organization summary only
6. Identify actual key risks for your project at this time
8. Use of GenAI tools
9.Team Lead Checklist

1. Executive Summary

- **Motivation**

  At San Francisco State University, tutoring is vital but often limited by rigid schedules, course mismatches, and disconnected systems. The demographic of the student base is unlike most academic institutions in the sense that many students are commuters, working parents, and graduate researchers. These groups will struggle to find tutoring help that fits their class topics and time constraints. *GatorTutors* was created to solve this problem by building a unified, student-driven tutoring platform tailored for the SFSU community.

- **What GatorTutors Does**
  GatorTutors connects SFSU students and verified peer tutors through an intuitive web interface that simplifies booking, communication, and session tracking. Students can search for tutors by course number, subject, or availability, and instantly book either online or in-person sessions. Tutors can manage their schedules, upload notes, and review session histories. The platform automatically handles notifications and reminders, ensuring smooth coordination for everyone. All interactions happen securely within the site—no third-party emails or payments required—making the system both safe and campus-compliant.
- **Why GatorTutors is Unique to SFSU**
  Unlike generic tutoring services such as Kumon.com or Mathnasium, GatorTutors is designed *for and by* SFSU students. It integrates directly with the university's course catalog, allowing tutoring to be matched at the class-code level rather than by vague subject areas. This means a Computer Science student can find a peer tutor who has already taken "CSC 415" or "CSC 648" at SFSU and understands the exact curriculum and professor expectations. The platform also verifies users through SFSU email credentials, maintaining a safe, closed network exclusive to our campus community. What's not better than someone who's been through that exact class giving exclusive tips and tricks specific to the professor's curriculum.
- **Value and Impact**
  By bridging flexibility, familiarity, and trust, GatorTutors extends the reach of academic support far beyond traditional tutoring centers. It empowers students to learn from one another, reduces administrative overhead, and creates new opportunities for paid peer mentorship.
- **About Our Team**
  We are team 14, a team composed of five dedicated SFSU computer science students. The team is led by William Wong, with James Barr and Ethan Niu leading frontend development, Uchenna Duru and Randy Chen driving backend architecture, and Cyrus Molina-Perez managing version control.

2. List of main data items and entities (Backend)

Key entities

1. Users (Base Entity)

- user_id - Unique identifier (mandatory, auto-generated)
- first_name - User's first name (mandatory, text)
- last_name - User's last name (mandatory, text)
- email - Login email address (mandatory, unique, text)
- password - Encrypted password (mandatory, text)
- role - User type: student, tutor, admin (mandatory, selection)
- profile_picture - Path to profile image (optional, text)
- bio - Personal description (optional, text, max 500 chars)
- phone_number - Contact number (optional, text)
- date_joined - Registration date (mandatory, auto-generated)
- last_login - Last access time (optional, timestamp)
- is_active - Account status (mandatory, boolean, default true)
- sfsu_id - SFSU student/staff ID (optional, text) [SFSU-specific]
- campus_location - On/off campus (optional, selection) [SFSU-specific]

2. Students (Extends Users)

- student_id - Links to Users table (mandatory, foreign key)
- grade_level - Freshman/Sophomore/Junior/Senior/Graduate (mandatory, selection)
- major - Academic major (mandatory, text)
- preferred_subjects - Subjects seeking help in (optional, text/JSON)
- learning_goals - Personal objectives (optional, text, max 1000 chars)
- gpa - Current GPA (optional, decimal 0.0-4.0)
- sfsu_college - College within SFSU (optional, selection) [SFSU-specific]

3. Tutors (Extends Users)

- tutor_id - Links to Users table (mandatory, foreign key)
- subjects_offered - Teaching subjects (mandatory, text/JSON)
- education_level - Bachelors/Masters/PhD/Other (mandatory, selection)
- certifications - Teaching credentials (optional, text)
- experience_years - Teaching experience (mandatory, integer, default 0)
- hourly_rate - Rate per hour in USD (mandatory, decimal)
- rating - Average rating 0.00-5.00 (mandatory, decimal, auto-calculated)
- total_reviews - Number of reviews (mandatory, integer, auto-calculated)

- teaching_mode - online/in-person/both (mandatory, selection)
- sfsu_verified - Verified SFSU tutor (optional, boolean) [SFSU-specific]
- department_approved - Department approval status (optional, boolean) [SFSU-specific]

4. Subjects

- subject_id - Unique identifier (mandatory, auto-generated)
- subject_name - Name of subject (mandatory, text)
- category - STEM/Liberal Arts/Business/Other (mandatory, selection)
- description - Subject details (optional, text, max 500 chars)
- difficulty_level - beginner/intermediate/advanced (optional, selection)
- sfsu_course_code - Official SFSU code like "CS 648" (optional, text) [SFSU-specific]
- department - SFSU department (optional, text) [SFSU-specific]
- is_active - Currently offered (mandatory, boolean, default true)

5. Courses (Group Learning)

- course_id - Unique identifier (mandatory, auto-generated)
- tutor_id - Teaching tutor (mandatory, foreign key)
- subject_id - Course subject (mandatory, foreign key)
- course_title - Name of course (mandatory, text, max 200 chars)
- description - Course details (mandatory, text)
- syllabus - Course outline document (optional, text/file path)
- difficulty_level - beginner/intermediate/advanced (mandatory, selection)
- price - Total cost in USD (mandatory, decimal)
- start_date - Course start (mandatory, date)
- end_date - Course end (mandatory, date)
- schedule - Meeting times (mandatory, text/JSON)
- max_students - Capacity limit (mandatory, integer, default 10)
- current_students - Enrolled count (mandatory, integer, auto-calculated)
- status - upcoming/ongoing/completed/cancelled (mandatory, selection)
- course_materials - Links to resources (optional, text/JSON)
- location - online/on-campus/off-campus (mandatory, selection) [SFSU-specific]

6. Sessions (Individual Tutoring)

- session_id - Unique identifier (mandatory, auto-generated)
- course_id - Related course if any (optional, foreign key)
- tutor_id - Session tutor (mandatory, foreign key)
- student_id - Session student (mandatory, foreign key)
- subject_id - Session topic (mandatory, foreign key)
- start_time - Session start (mandatory, datetime)
- end_time - Session end (mandatory, datetime)

- duration_minutes - Length in minutes (mandatory, integer, auto-calculated)
- status - scheduled/completed/cancelled/no-show (mandatory, selection)
- session_type - one-on-one/group (mandatory, selection)
- session_link - Video meeting URL (optional, text)
- session_notes - Tutor's notes (optional, text)
- student_feedback - Student's notes (optional, text)
- recording_url - Session recording (optional, text)
- location - Physical location if in-person (optional, text)

7. Bookings (Session Requests)

- booking_id - Unique identifier (mandatory, auto-generated)
- student_id - Requesting student (mandatory, foreign key)
- tutor_id - Requested tutor (mandatory, foreign key)
- subject_id - Topic needed (mandatory, foreign key)
- requested_date - Preferred date (mandatory, date)
- requested_time - Preferred time (mandatory, time)
- duration_requested - Length in minutes (mandatory, integer)
- urgency - normal/urgent (mandatory, selection, default normal)
- status - pending/confirmed/declined/cancelled (mandatory, selection)
- message_to_tutor - Student's request details (optional, text, max 500 chars)
- tutor_response - Tutor's reply (optional, text)
- alternative_times - Other available slots (optional, text/JSON)

8. Enrollments

- enrollment_id - Unique identifier (mandatory, auto-generated)
- course_id - Enrolled course (mandatory, foreign key)
- student_id - Enrolled student (mandatory, foreign key)
- enrollment_date - Join date (mandatory, timestamp, auto-generated)
- progress_percentage - Course completion 0-100 (mandatory, integer, default 0)
- completion_status - enrolled/in_progress/completed/dropped (mandatory, selection)
- completion_date - Finish date (optional, timestamp)
- certificate_issued - Completion certificate (optional, boolean)

9. Transactions (Payments)

- payment_id - Unique identifier (mandatory, auto-generated)
- payer_id - Person paying (mandatory, foreign key to Users)
- receiver_id - Person receiving (mandatory, foreign key to Users)
- session_id - Related session (optional, foreign key)
- course_id - Related course (optional, foreign key)
- amount - Payment in USD (mandatory, decimal)
- payment_method - credit_card/debit_card/paypal/stripe (mandatory, selection)

- payment_status - pending/completed/failed/refunded (mandatory, selection)
- transaction_date - Payment time (mandatory, timestamp, auto-generated)
- transaction_reference - External payment ID (optional, text)
- refund_reason - If refunded (optional, text)

10. Reviews

- review_id - Unique identifier (mandatory, auto-generated)
- student_id - Reviewing student (mandatory, foreign key)
- tutor_id - Reviewed tutor (mandatory, foreign key)
- course_id - Related course (optional, foreign key)
- session_id - Related session (optional, foreign key)
- rating - Score 1-5 stars (mandatory, integer)
- comment - Written review (optional, text, max 1000 chars)
- date_posted - Review date (mandatory, timestamp, auto-generated)
- is_verified - Verified attendance (mandatory, boolean, auto-calculated)
- is_visible - Public visibility (mandatory, boolean, default true)
- helpful_count - Users who found helpful (optional, integer, default 0)

11. Messages

- message_id - Unique identifier (mandatory, auto-generated)
- sender_id - Sending user (mandatory, foreign key)
- receiver_id - Receiving user (mandatory, foreign key)
- conversation_id - Thread identifier (mandatory, text)
- message_text - Message content (mandatory, text)
- attachment_url - File attachment (optional, text)
- timestamp - Send time (mandatory, timestamp, auto-generated)
- is_read - Read status (mandatory, boolean, default false)
- is_deleted - Soft delete (mandatory, boolean, default false)

12. Availability

- availability_id - Unique identifier (mandatory, auto-generated)
- tutor_id - Tutor's schedule (mandatory, foreign key)
- day_of_week - Monday-Sunday (mandatory, selection)
- start_time - Available from (mandatory, time)
- end_time - Available until (mandatory, time)
- is_recurring - Weekly repeat (mandatory, boolean, default true)
- specific_date - For one-time availability (optional, date)
- is_booked - Already reserved (mandatory, boolean, default false)

13. Notifications

- notification_id - Unique identifier (mandatory, auto-generated)
- user_id - Recipient user (mandatory, foreign key)

    - type - booking/payment/message/course/session/review (mandatory, selection)
    - title - Notification title (mandatory, text, max 100 chars)
    - content - Notification details (mandatory, text)
    - related_id - ID of related item (optional, integer)
    - priority - low/normal/high (mandatory, selection, default normal)
    - timestamp - Created time (mandatory, timestamp, auto-generated)
    - is_read - Read status (mandatory, boolean, default false)
    - is_email_sent - Email sent status (optional, boolean, default false)

14. SFSU-Specific Entities

- Professor Verifications [SFSU-specific]
    - verification_id - Unique identifier (mandatory, auto-generated)
    - professor_email - SFSU professor email (mandatory, text)
    - professor_name - Professor full name (mandatory, text)
    - department - Academic department (mandatory, text)
    - verified_tutor_id - Approved tutor (mandatory, foreign key)
    - verification_date - Approval date (mandatory, timestamp)
    - comments - Professor notes (optional, text)
- Campus Resources [SFSU-specific]
    - resource_id - Unique identifier (mandatory, auto-generated)
    - resource_name - Name of resource (mandatory, text)
    - resource_type - tutoring_center/study_room/library (mandatory, selection)
    - building - Campus building (mandatory, text)
    - room_number - Location (optional, text)
    - hours_of_operation - Operating hours (mandatory, text)
    - contact_info - Phone/email (optional, text)
    - is_available - Current availability (mandatory, boolean)

3. Functional Requirements (Backend)

5.1 Unregistered User Requirements

1. The system shall allow unregistered users to browse available tutoring subjects and courses offered at SFSU.
2. The system shall allow unregistered users to view tutor profiles including their qualifications, ratings, and availability.
3. The system shall allow unregistered users to search for tutors by subject, course number, availability, and rating.
4. The system shall require users to create an account and verify SFSU affiliation using verification code emails sent from a gmail account before booking tutoring sessions.

5.2 Registered User Requirements

1. The system shall allow registered SFSU students to book tutoring sessions with available tutors for specific SFSU courses.
2. The system shall allow students to cancel at least 24 hours in advance.
3. The system shall allow students to reschedule tutoring sessions at least 24 hours in advance.
4. The system shall allow students to view their upcoming and past tutoring sessions in a dashboard.
5. The system shall allow students to rate and review tutors after completing a session.
6. The system shall allow students to save favorite tutors for quick booking access.
7. The system shall send automated email notifications to students for session confirmations, reminders, and cancellations.
8. The system shall allow students to access session notes and materials uploaded by tutors after sessions are completed.
9. The system shall allow students to filter tutors by rating, availability, and specific SFSU course expertise.
10. The system shall allow registered SFSU tutors to create and manage their availability schedules for tutoring sessions.
11. The system shall allow tutors to specify which SFSU courses and subjects they are qualified to tutor.
12. The system shall allow tutors to upload session notes, study materials, and resources for their students.
13. The system shall allow tutors to accept or decline booking requests from students.
14. The system shall allow tutors to view their session history and student feedback.

5.3 Admin Requirements

1. The system shall allow administrators to verify and approve tutor credentials and SFSU student enrollment status.
2. The system shall allow administrators to generate reports on tutoring session metrics, tutor performance, and platform usage.
3. The system shall allow administrators to manage user accounts including suspension or removal for policy violations.

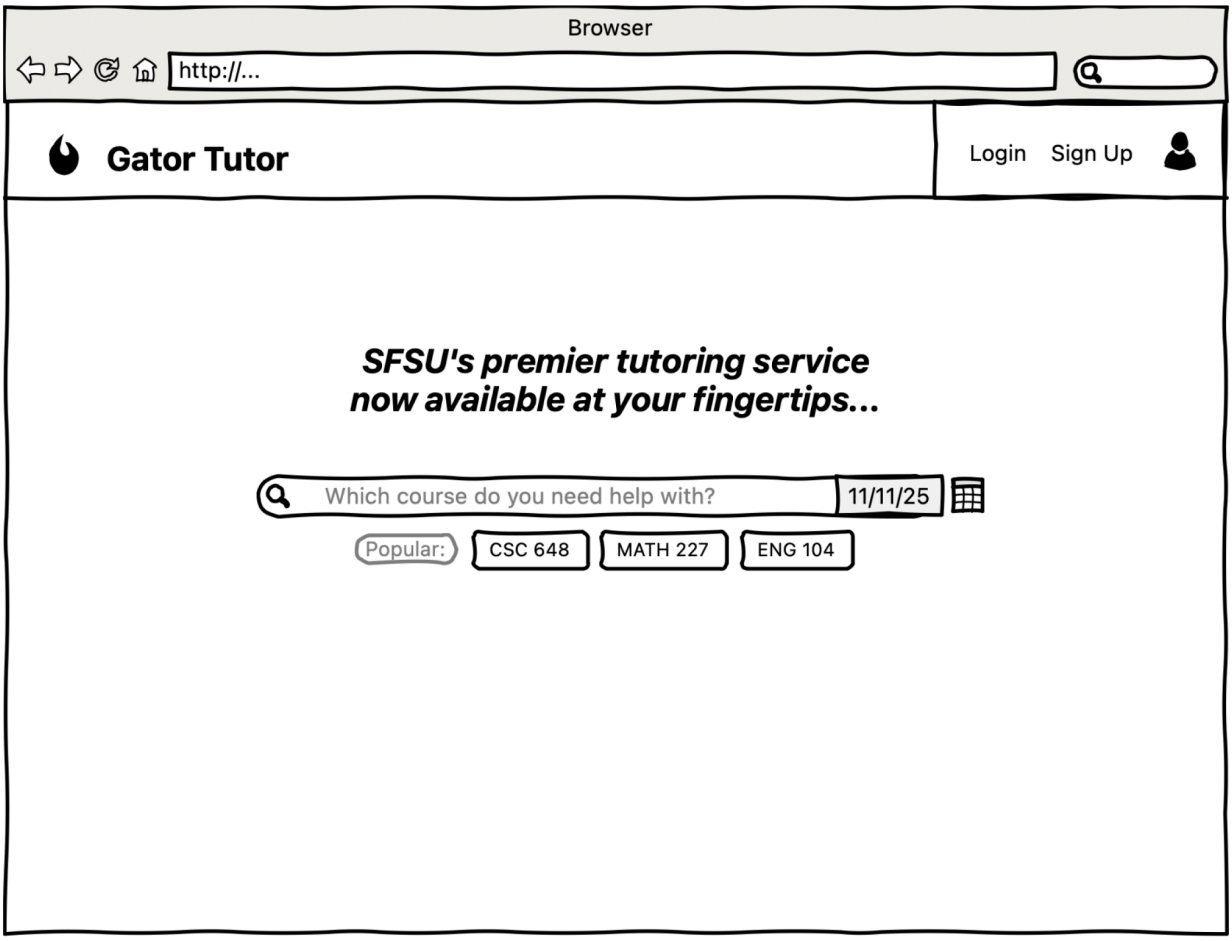
5.4 SFSU-Specific Requirement(s)

1. The system shall integrate with SFSU's course catalog to automatically populate available courses and current semester offerings for tutor specialization selection

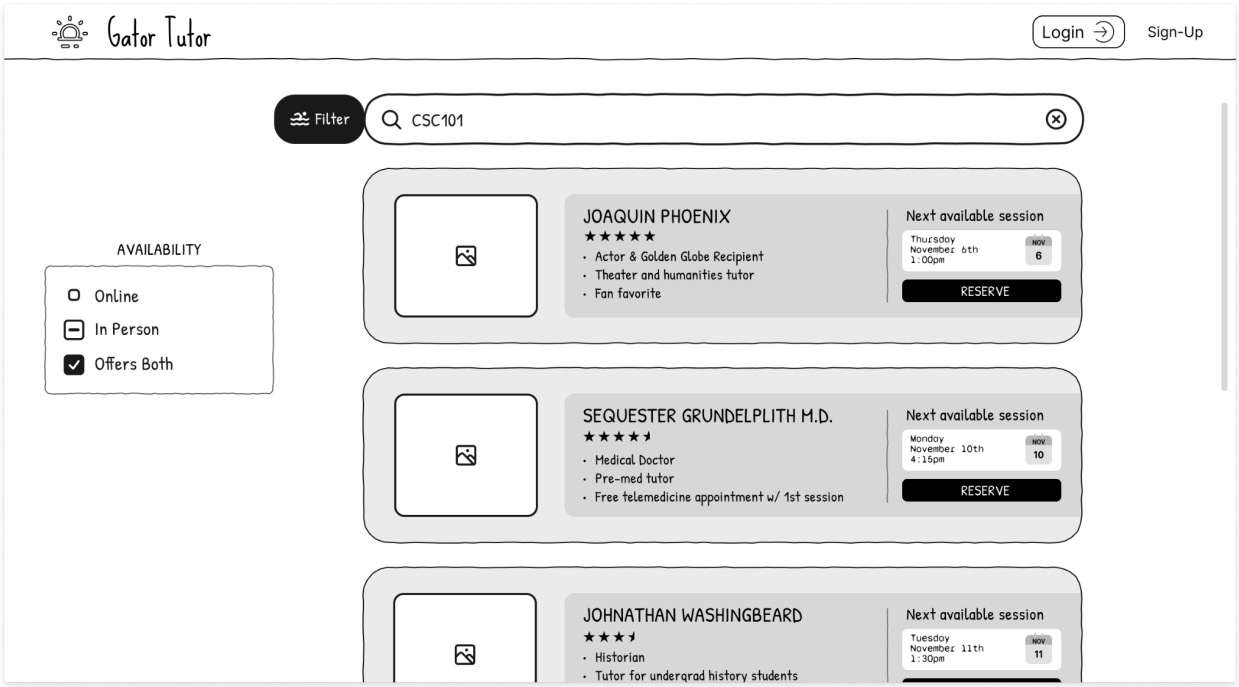
4. UI Storyboards for each main use case (Frontend)

Returning Student Use Case:

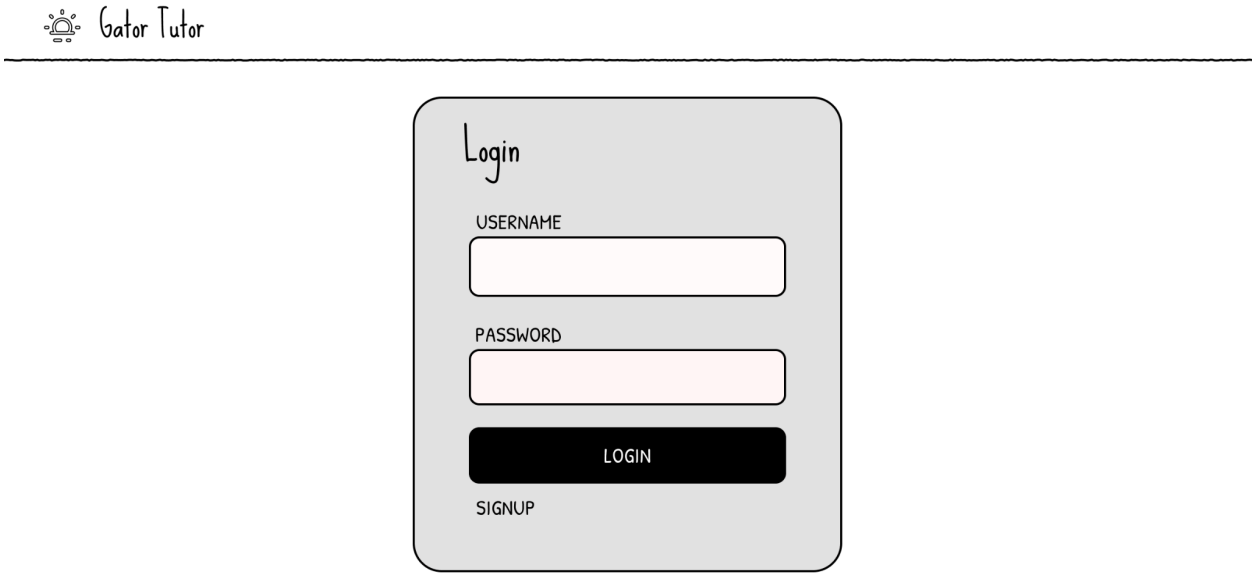
landing page -> search results -> book session-> login-> attend session -> check notes later



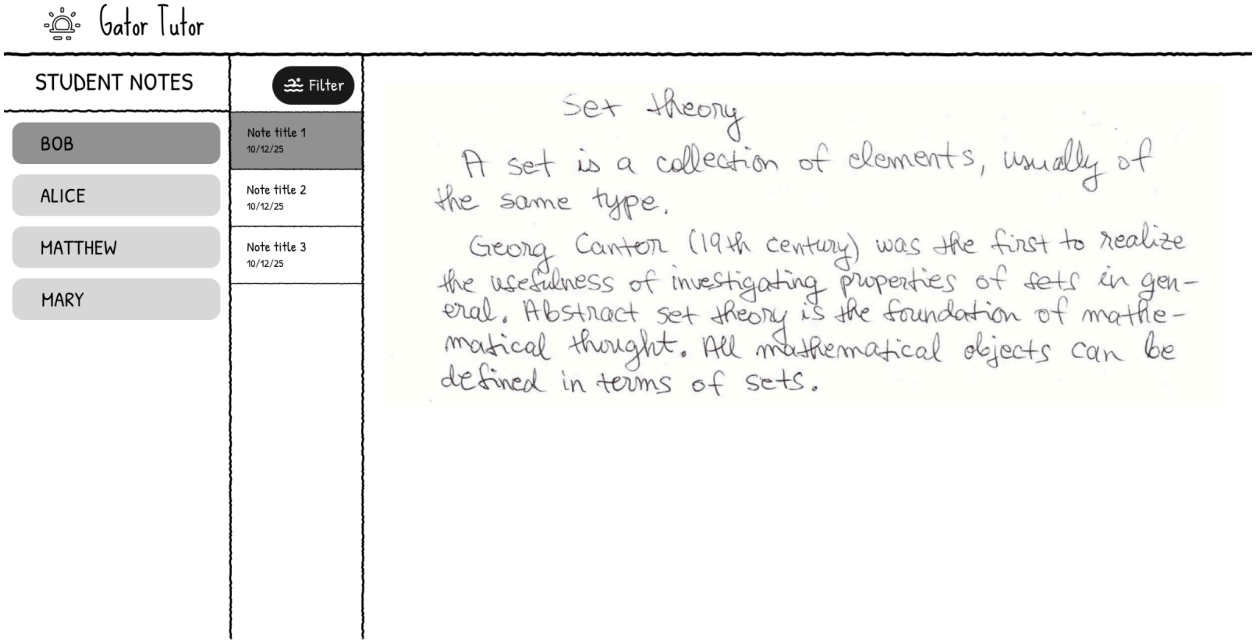
Search results



Login Prompt



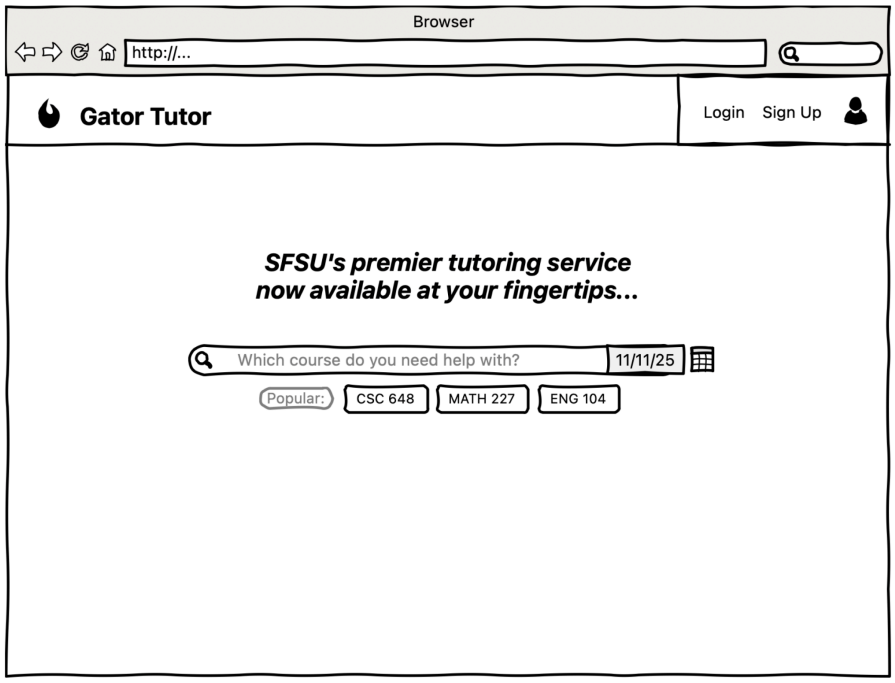
Notes and shared files



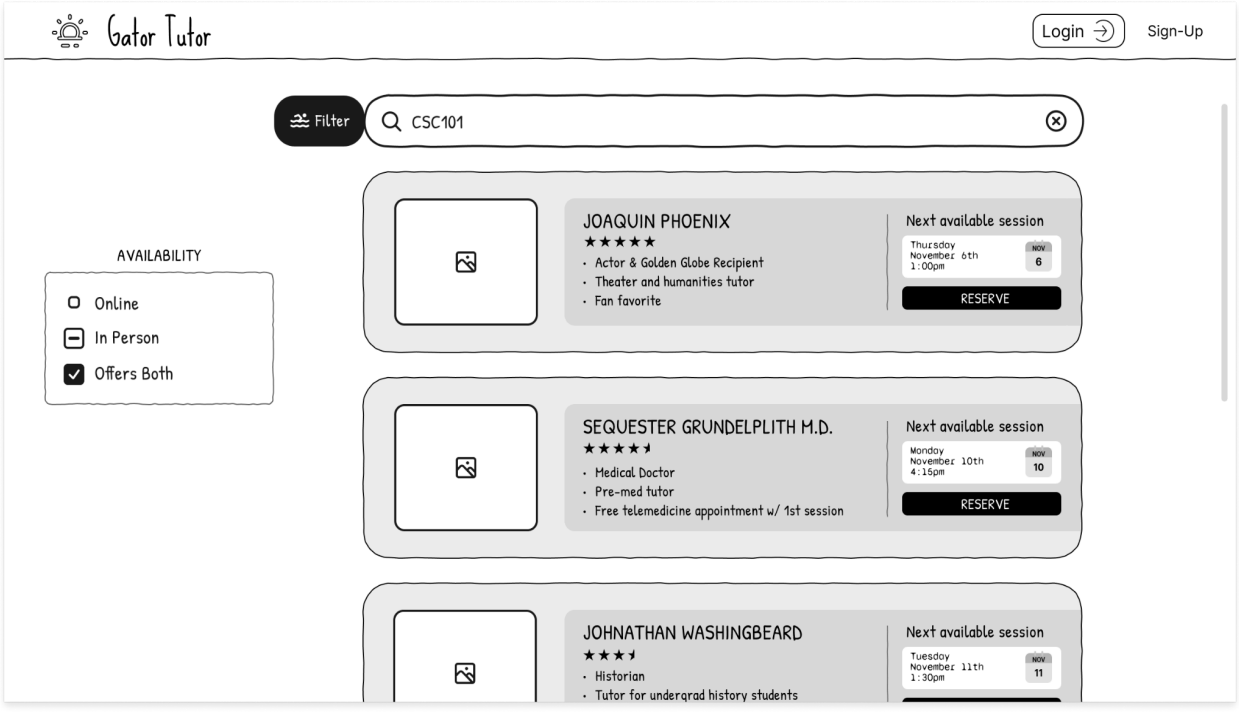
New Student Use Case

Home -> search results -> login -> signup -> auth -> book

Home

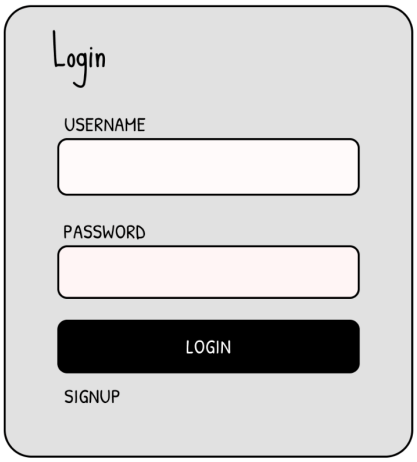


Results



Login

Gator Tutor



Sign Up

Welcome!

Which account would you like to create?

Student

- Can book sessions with available tutors
- Instant sign-up

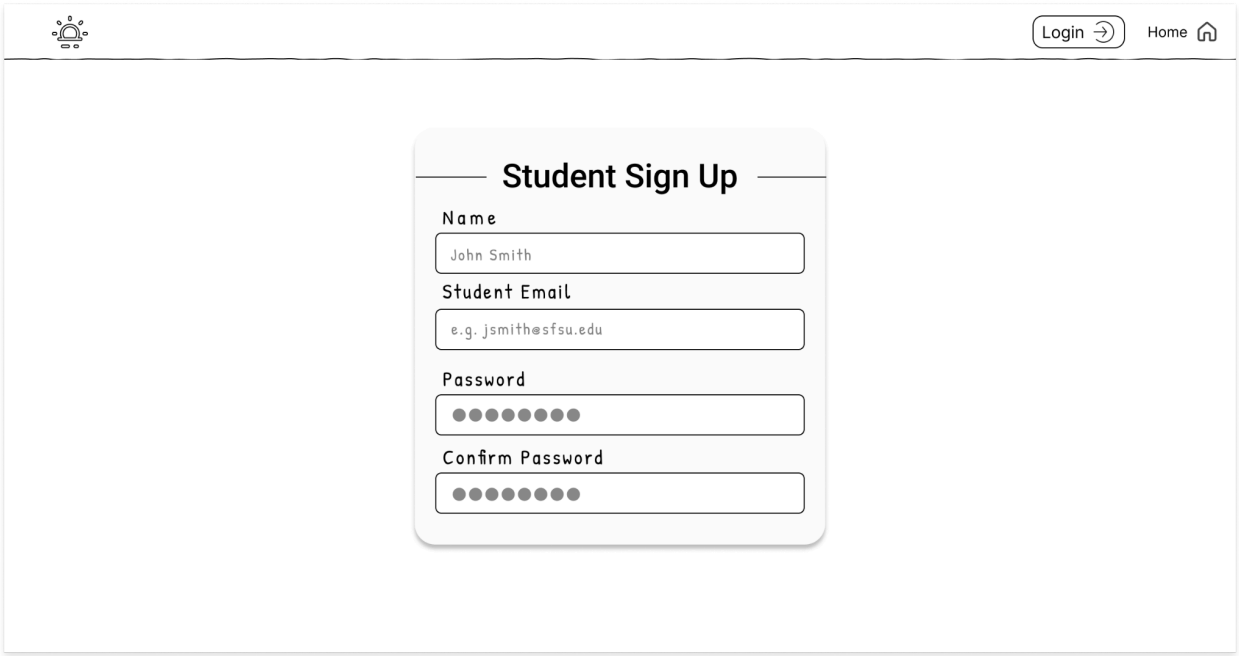
Register

Tutor

- Can offer new sessions
- Requires verification

Register

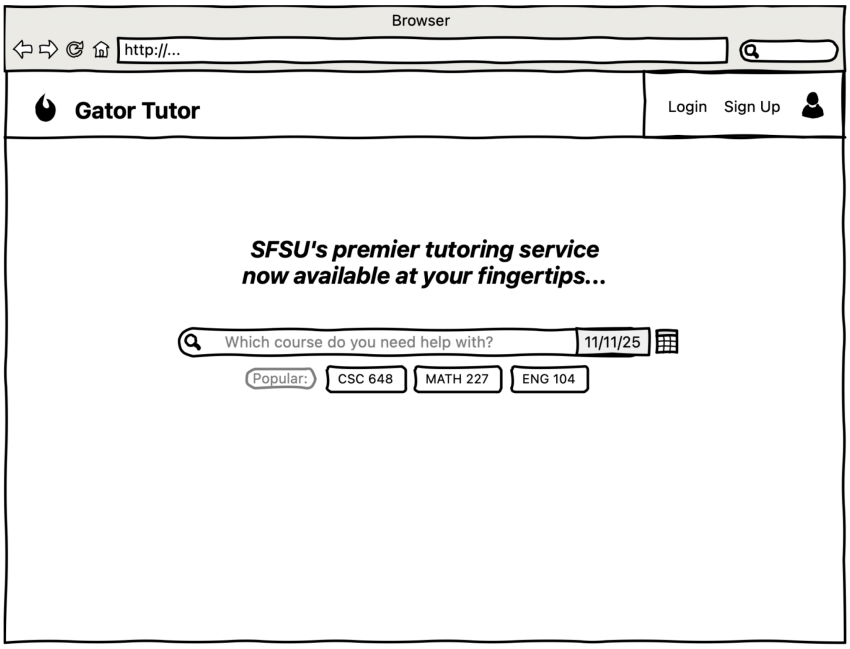
Student Account Sign Up



Auth



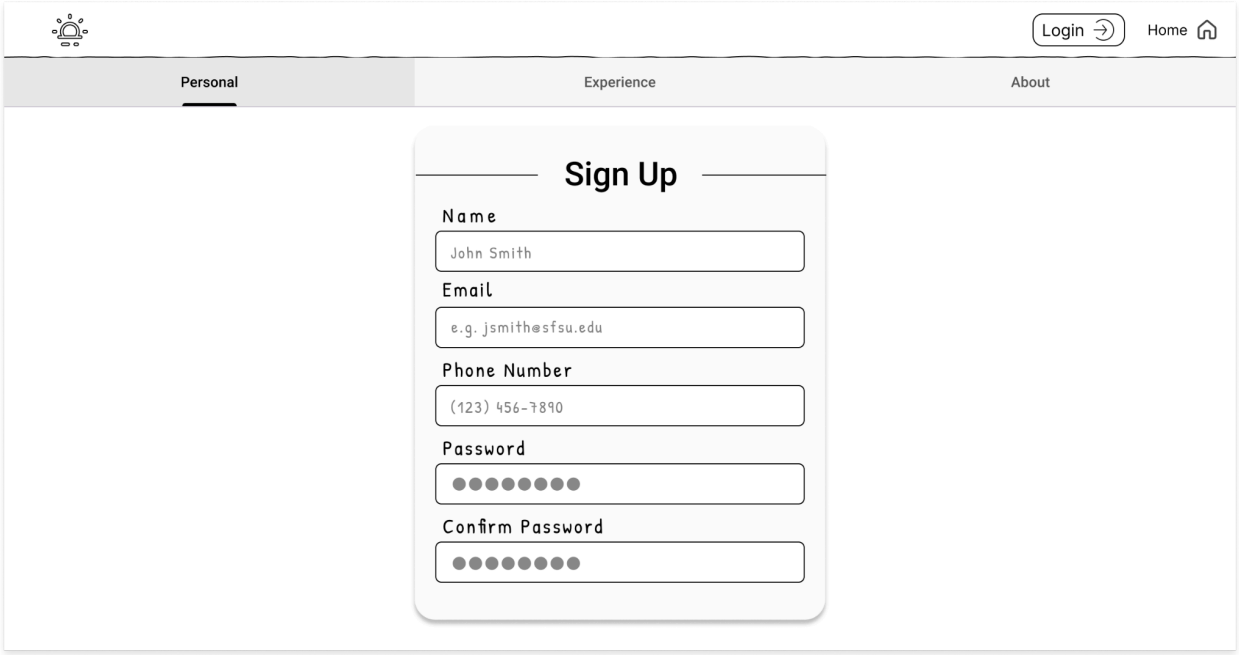
Tutor Sign Up

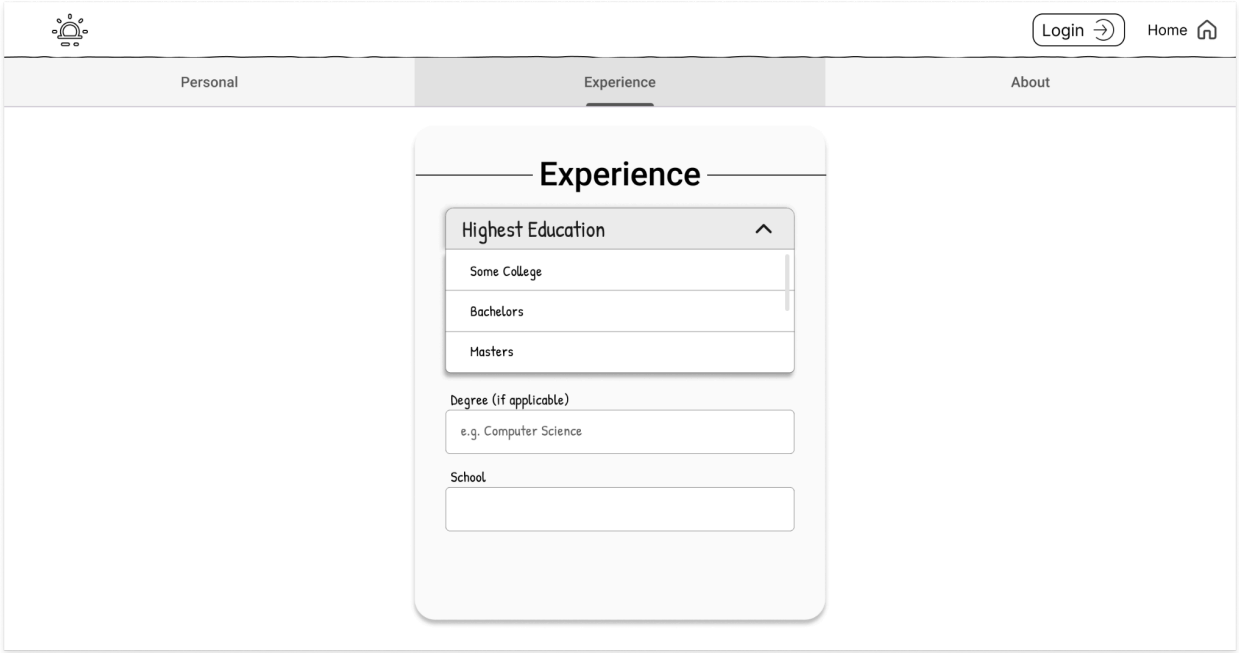


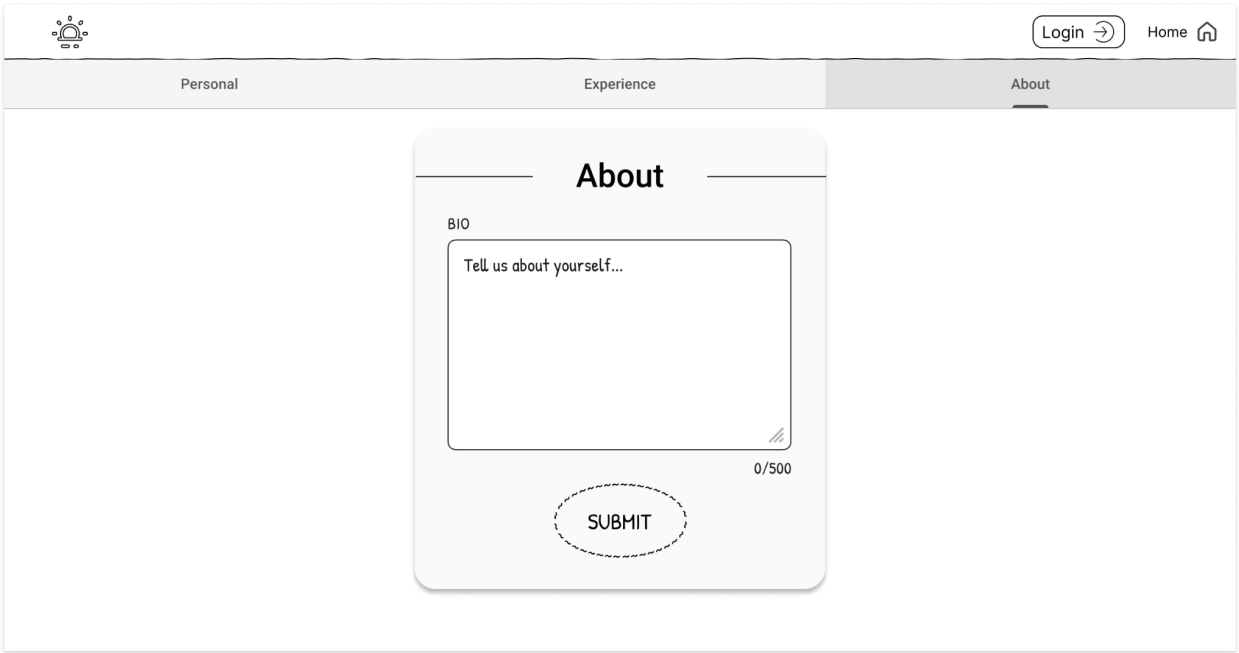
Sign Up

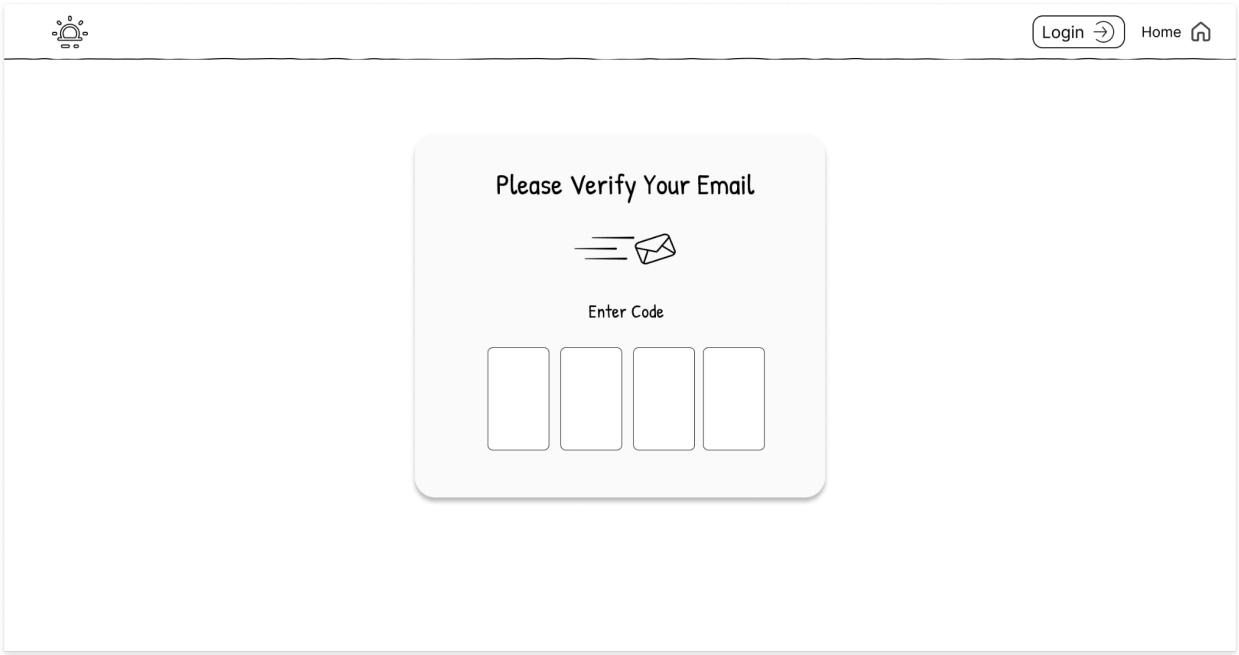


Create account






Login
Home
Personal
Experience
About
About
BIO
Tell us about yourself...
0/500
SUBMIT



5. High level Architecture, Database Organization summary only (Backend)

Overall Architecture (MVC Pattern):

Model (M): Each table in MySQL is mapped to a simple model file that runs basic SQL queries (SELECT, INSERT, UPDATE, DELETE) using the mysql2 package.

View (V): HTML pages with minimal CSS and JS. Only handles basic forms and lists (e.g., login page, tutor list, course list).

Controller (C): Express routes that call model functions and return JSON or render HTML pages. Each controller handles one feature area (e.g., users, courses, sessions).

Users

- user_id (Primary key, int, auto-increment)
- first _name (varchar(50)

- last_name (varchar(50)
- email (varchar(100)
- password (varchar(255)
- role (
- profile_picture (varchar(50)
- bio (varchar(50)
- phone_number (optional, text)
- date_joined (mandatory, auto-generated)
- last_login (optional, timestamp)
- is_active (mandatory, boolean, default true)
- sfsu_id (optional, text)
- campus_location (optional, selection)

Students

- student_id (Primary Key, int, references users.user_id)
- grade_level (enum: 'freshman', 'sophomore', 'junior', 'senior', 'graduate')
- major (varchar(100), student's declared major)
- preferred_subjects (text, optional list of subjects the student is interested in)
- learning_goals (varchar(1000), optional description of academic or personal goals)
- gpa (decimal(3,2), optional student GPA)
- sfsu_college (varchar(100), optional college affiliation, e.g., College of Science & Engineering)

Tutors

- tutor_id (Primary Key, int, references users.user_id)
- subjects_offered (text, list of subjects the tutor can teach)
- education_level (enum: 'bachelors', 'masters', 'phd', 'other')
- certifications (text, optional tutor certifications or credentials)
- experience_years (int, number of years of tutoring or teaching experience)
- hourly_rate (decimal(10,2), tutor's hourly charge in USD)
- rating (decimal(3,2), average rating based on student reviews, default 0.00)
- total_reviews (int, total number of reviews received, default 0)
- teaching_mode (enum: 'online', 'in_person', 'both')
- sfsu_verified (tinyint(1), indicates if tutor is verified by SFSU faculty)
- department_approved (tinyint(1), marks if tutor has departmental approval)

Subjects

- subject_id (Primary Key, int, auto-increment)
- subject_name (varchar(100), name of the subject or course)
- category (enum: 'STEM', 'Liberal Arts', 'Business', 'Other')
- description (varchar(500), optional short summary of the subject)
- difficulty_level (enum: 'beginner', 'intermediate', 'advanced')
- sfsu_course_code (varchar(20), optional university course reference)
- department (varchar(100), academic department offering the subject)

- is_active (tinyint(1), default 1, marks if the subject is active)

Courses

- course_id (Primary Key, int, auto-increment)
- tutor_id (int, Foreign Key → users.user_id)
- subject_id (int, Foreign Key → subjects.subject_id)
- course_title (varchar(200), name of the course)
- description (text, detailed overview of the course)
- syllabus (varchar(255), optional file path or URL to syllabus)
- difficulty_level (enum: 'beginner', 'intermediate', 'advanced')
- price (decimal(10,2), course fee in USD)
- start_date (date, start of the course)
- end_date (date, end of the course)
- schedule (text, details of session times or weekly plan)
- max_students (int, default 10, maximum number of allowed students)
- current_students (int, default 0, number of currently enrolled students)
- status (enum: 'upcoming', 'ongoing', 'completed', 'cancelled')
- course_materials (text, optional list or links to course resources)
- location (enum: 'online', 'on_campus', 'off_campus')
- created_at (timestamp, default CURRENT_TIMESTAMP)

Sessions

- session_id (Primary Key, int, auto-increment)
- course_id (int, Foreign Key → courses.course_id, optional link to a course)
- tutor_id (int, Foreign Key → users.user_id)
- student_id (int, Foreign Key → users.user_id)
- subject_id (int, Foreign Key → subjects.subject_id)
- start_time (datetime, scheduled session start)
- end_time (datetime, scheduled session end)
- duration_minutes (int, stored generated column for total session length)
- status (enum: 'scheduled', 'completed', 'cancelled', 'no_show')
- session_type (enum: 'one_on_one', 'group')
- session_link (varchar(255), optional online meeting link)
- session_notes (text, optional notes entered by the tutor or admin)
- student_feedback (text, optional feedback from the student)
- recording_url (varchar(255), optional link to recorded session)
- location (varchar(255), optional physical or virtual location)
- created_at (timestamp, default CURRENT_TIMESTAMP)

Bookings

- booking_id (Primary Key, int, auto-increment)
- student_id (int, Foreign Key → users.user_id)

- tutor_id (int, Foreign Key → users.user_id)
- subject_id (int, Foreign Key → subjects.subject_id)
- requested_date (date, not null)
- requested_time (time, not null)
- duration_requested (int, not null)
- urgency (enum('normal','urgent'), default 'normal')
- status (enum('pending','confirmed','declined','cancelled'), not null)
- message_to_tutor (varchar(500), nullable)
- tutor_response (text, nullable)
- alternative_times (text, nullable)
- created_at (timestamp, default CURRENT_TIMESTAMP)
- updated_at (timestamp, default CURRENT_TIMESTAMP on update CURRENT_TIMESTAMP)

Enrollments

- enrollment_id (Primary Key, int, auto-increment)
- course_id (int, Foreign Key → courses.course_id)
- student_id (int, Foreign Key → users.user_id)
- enrollment_date (timestamp, default CURRENT_TIMESTAMP)
- progress_percentage (int, default 0, tracks student's course progress)
- completion_status (enum: 'enrolled', 'in_progress', 'completed', 'dropped')
- completion_date (timestamp, optional, set when a course is completed)
- certificate_issued (tinyint(1), default 0, indicates if a completion certificate was granted)

Transactions

- payment_id (Primary Key, int, auto-increment)
- payer_id (int, Foreign Key → users.user_id, the student making the payment)
- receiver_id (int, Foreign Key → users.user_id, the tutor receiving the payment)
- session_id (int, Foreign Key → sessions.session_id, optional link to the tutoring session)
- course_id (int, Foreign Key → courses.course_id, optional related course)
- amount (decimal(10,2), payment amount in USD)
- payment_method (enum: 'credit_card', 'debit_card', 'paypal', 'stripe')
- payment_status (enum: 'pending', 'completed', 'failed', 'refunded')
- transaction_date (timestamp, default CURRENT_TIMESTAMP)
- transaction_reference (varchar(100), optional payment gateway reference or ID)
- refund_reason (text, optional note if a payment is refunded)

Reviews

- review_id (Primary Key, int, auto-increment)
- student_id (int, Foreign Key → users.user_id)
- tutor_id (int, Foreign Key → users.user_id)
- course_id (int, Foreign Key → courses.course_id)
- session_id (int, Foreign Key → sessions.session_id)
- rating (int, 1–5 scale)
- comment (varchar(1000))

- date_posted (timestamp, default CURRENT_TIMESTAMP)
- is_verified (tinyint(1), default 0)
- is_visible (tinyint(1), default 1)
- helpful_count (int, default 0)

Messages

- message_id (Primary Key, int, auto-increment)
- sender_id (int, Foreign Key → users.user_id)
- receiver_id (int, Foreign Key → users.user_id)
- conversation_id (varchar(100), groups messages in the same chat)
- message_text (text, message content)
- attachment_url (varchar(255), optional file or image link)
- timestamp (timestamp, default CURRENT_TIMESTAMP)
- is_read (tinyint(1), default 0, marks if message has been read)
- is_deleted (tinyint(1), default 0, marks if message was deleted by user)

Availability

- availability_id (Primary Key, int, auto-increment)
- tutor_id (int, Foreign Key → users.user_id)
- day_of_week (enum: 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')
- start_time (time, tutor's available start time)
- end_time (time, tutor's available end time)
- is_recurring (tinyint(1), default 1, marks if availability repeats weekly)
- specific_date (date, optional specific availability date)
- is_booked (tinyint(1), default 0, marks if the slot has been booked)

Notifications

- notification_id (Primary Key, int, auto-increment)
- user_id (int, Foreign Key → users.user_id)
- type (enum: 'booking', 'payment', 'message', 'course', 'session', 'review')
- title (varchar(100), short summary of the notification)
- content (text, detailed message body)
- related_id (int, optional reference to related record such as session_id or review_id)
- priority (enum: 'low', 'normal', 'high', default 'normal')
- timestamp (timestamp, default CURRENT_TIMESTAMP)
- is_read (tinyint(1), default 0, marks if the notification was opened)
- is_email_sent (tinyint(1), default 0, marks if an email notification was also sent)

Professor_verifications

- verification_id (Primary Key, int, auto-increment)
- professor_email (varchar(100), email of the verifying professor)
- professor_name (varchar(100), full name of the verifying professor)

- department (varchar(100), professor's academic department)
- verified_tutor_id (int, Foreign Key → users.user_id)
- verification_date (timestamp, default CURRENT_TIMESTAMP)
- comments (text, optional remarks from the professor)

Campus_resourses

- resource_id (Primary Key, int, auto-increment)
- resource_name (varchar(100), name of the resource)
- resource_type (enum: 'tutoring_center', 'study_room', 'library')
- building (varchar(100), building where the resource is located)
- room_number (varchar(20), optional room identifier)
- hours_of_operation (varchar(255), operating hours or schedule)
- contact_info (varchar(255), optional phone number or email)
- is_available (tinyint(1), default 1, indicates if the resource is open for use)

Media Storage:

Profile pictures or uploads are stored as files in an /uploads folder.
The database only stores the file path or filename (no BLOBs).

Search and Filter:

Search is intentionally simple — users can search by class name only.
Example SQL:
SELECT * FROM courses WHERE course_title LIKE '%math%';

This keeps implementation easy but introduces a non-technical ambiguity issue:
Certain classes share similar names but serve different majors. For example, "Discrete Math" is required for Computer Science majors but not for Mathematics majors.
As a result, a search for "Math" may return both CS-specific and Math-department courses.
This isn't a system error — it's a naming overlap issue. Future versions may fix this by tagging each course with a department or major requirement column.

Algorithms/ Special Processes:

All operations are basic CRUD. No automatic ranking, scheduling, or rating aggregation is implemented at this stage.

Tools and Frameworks:

- Node.js + Express — backend routing
- MySQL + mysql2 — database connection
- bcrypt — password hashing
- HTML/CSS/JS — simple frontend

Summary:

The backend focuses only on essential CRUD functions and class-based search.
It avoids complex automation or logic so development stays lightweight, functional, and easy to expand later.
Known naming overlaps (e.g., “Discrete Math”) are noted but handled manually until department tagging is added.

6. Identify actual key risks for your project at this time (Frontend + Team Lead)

1. Skills Risk – Limited Backend Experience
Some team members are new to Node.js and MySQL, leading to slower development and potential bugs in database queries.
Mitigation: Pair program on core features, follow tutorials/documentation, and divide tasks by strength (front-end vs back-end). Conduct code reviews to share knowledge.

2. Schedule Risk – Tight Timeline for Feature Completion
The team has limited time to implement authentication, tutor search, and scheduling before the deadline.
Mitigation: Prioritize MVP features first (login, search, booking), defer non-essential UI improvements to later iterations, and track progress weekly using a task board (Trello or GitHub Projects).

3. Technical Risk – Server and Database Integration Issues

Unfamiliarity with connecting Node.js to MySQL and handling async queries could cause errors or crashes.
Mitigation: Test database operations in small modules, use connection pooling and error handling, and create mock data for local testing before deployment.

4. Teamwork Risk – Uneven Workload Distribution
Some members are contributing more to coding while others focus mainly on design or documentation.
Mitigation: Assign clear weekly deliverables per member and hold short sync-ups to balance workload and encourage cross-learning.

5. Legal/Content Risk – Use of External Libraries and Assets
Potential use of icons, CSS frameworks, or images without proper licenses.
Mitigation: Use only open-source MIT / Apache licensed packages and royalty-free assets (Unsplash, FontAwesome, Bootstrap CDN) with credit when required.

8. Use of GenAI tools (Team Lead)

The use of AI was primarily in creating this document and making sure grammar and other wordings were done correctly and in an efficient manner. The only AI used was chatGPT.

9.Team Lead Checklist (Team Lead)

Checklist Item:	Status:
So far all team members are fully engaged and attending team sessions when required.	DONE
Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing	DONE
Team reviewed suggested resources before drafting Milestone 2	DONE
Team lead checked Milestone 2 document for quality, completeness, formatting and compliance with instructions before the submission	DONE
Team lead ensured that all team members read the final Milestone 2 document and agree/understand it before submission	DONE
Team shared and discussed experience with GenAI tools among themselves	DONE